

Documentação Técnica — Simulador de Escalonamento de Tarefas

Esta documentação descreve como o simulador funciona internamente.

1. Visão geral

O simulador reproduz o escalonamento de tarefas num processador único. Implementa seis políticas — FCFS, Round-Robin, SJF, SRTF, prioridade cooperativa (PRIOc) e prioridade preemptiva (PRIOp) — todas com custo de troca de contexto configurável, e o Round-Robin também com quantum configurável. Para cada uma, calcula o tempo de turnaround (`tt`), o tempo de espera (`tw`) e o tempo até a primeira execução de cada tarefa, além de desenhar um diagrama de tempo (formato Gantt) mostrando o que aconteceu instante a instante.

Sob PRIOp, o simulador também trata recurso de uso exclusivo: uma tarefa pode declarar que precisa de um recurso durante parte da própria execução, e o simulador reproduz o fenômeno da inversão de prioridade (uma tarefa de prioridade baixa segurando o processador — indiretamente — de uma de prioridade alta), junto com os dois mecanismos clássicos de correção, herança e teto de prioridade. Sob PRIOc, implementa envelhecimento de prioridade, que elimina a inanição de tarefas de prioridade baixa.

2. Separação entre política e mecanismo

Cada algoritmo (`algoritmos/fcfs.py`, `sjf.py`, `srtf.py`, `round_robin.py`, `prioc.py`, `priop.py`) implementa o próprio laço de simulação por inteiro. A **política** de cada um (qual tarefa roda a seguir, quando ela é interrompida) está embutida nesse laço.

O que é compartilhado entre todos — o **mecanismo** — é o contrato de entrada, saída e o pós-processamento:

- Todo algoritmo recebe `tarefas: list[Tarefa]` mais os parâmetros que usa, e devolve um `ResultadoSimulacao`.
- Internamente, todo algoritmo monta a mesma estrutura intermediária: `periodos_por_tarefa: dict[int, list[Periodo]]` — um registro cru de tudo que aconteceu (execução, troca de contexto, bloqueio), por tarefa.
- Esse dicionário é entregue a `algoritmos/_montar_resultado.montar_resultado`, que calcula `tt/tw/1ª execução/médias/eficiência` a partir dele — igual pra qualquer algoritmo, é o único lugar que sabe fazer essa conta.
- `algoritmos/_derivar_grafico.py` deriva, a partir do mesmo `periodos_por_tarefa`, os dados que só o gráfico usa (esperas, intervalos de posse de recurso, instantes de preempção) — não influenciam nenhuma métrica, só o desenho.

Ou seja: a política de cada algoritmo não sabe nada sobre como suas decisões viram números ou desenho; ela só precisa produzir uma lista de `Periodo` coerente por tarefa.

3. Diagrama de módulos

Ao apertar o botão gerar gráfico:

```
Tarefa (entrada, cadastrada na tela)
|
v
algoritmo -- fcfs/sjf/srtf/round_robin/prioc/priop.py
|
v
periodos_por_tarefa: dict[id, list[Periodo]]
|
v
_montar_resultado.montar_resultado (mecanismo comum)
|
+--> métricas por tarefa + médias + parâmetros -----> tabela-resumo (chart_summary_table.py)
|
v
ResultadoSimulacao (tarefas: dict[id, TarefaResultado], ...)
|
v
_derivar_grafico.py (esperas, recursos_em_uso, preempcoes)
|
v
chart.py (desenha o diagrama de tempo) + chart_legend.py (legenda)
```

4. Estrutura de uma tarefa

Tarefa (models/tarefa.py):

Campo	Tipo	Função
id	inteiro	Identificador da tarefa — a posição em que foi cadastrada na tela (1-based)
chegada	inteiro	Instante em que a tarefa surge
tp	inteiro	Tempo de processamento total demandado
prioridade	inteiro	Prioridade da tarefa — maior valor, mais prioritária . Só é relevante sob PRIOc/PRIOp
recursos	lista de Recurso	Recursos de uso exclusivo que a tarefa disputa. Só é relevante sob PRIOp

Recurso (mesmo arquivo) — o vínculo de uma tarefa com um recurso de uso exclusivo:

Campo	Tipo	Função
<code>id</code>	inteiro	Identificador do recurso, compartilhado entre todas as tarefas que o disputam
<code>inicio</code>	inteiro	Depois de quanto tempo de processamento próprio da tarefa ela solicita o recurso
<code>duracao</code>	inteiro	Por quanto tempo de processamento próprio ela mantém o recurso, uma vez obtido

5. Interface dos módulos

Algoritmos (`algoritmos/`) — todos com a assinatura (`tarefas: list[Tarefa], ctx_time: float, ...`) -> `ResultadoSimulacao`:

- `fcfs(tarefas, ctx_time)` — despacha na ordem de chegada, sem preempção.
- `sjf(tarefas, ctx_time)` — despacha a de menor `tp` entre as já chegadas, sem preempção.
- `srtf(tarefas, ctx_time)` — como o SJF, mas reavalia a cada nova chegada e preempta se a recém-chegada tiver `tp` restante menor.
- `round_robin(tarefas, ctx_time, quantum)` — cada tarefa roda no máximo `quantum` segundos por vez, voltando pro fim da fila se não terminar.
- `prioc(tarefas, ctx_time, alpha=None)` — despacha por prioridade, sem preempção; com envelhecimento (`alpha`).
- `priop(tarefas, ctx_time, protocolo=None)` — despacha por prioridade, preemptivo; trata recurso de uso exclusivo e o protocolo de correção (`None`, `"Herança"` ou `"Teto"`).

Mecanismo comum (`algoritmos/_montar_resultado.py`):

- `montar_resultado(tarefas, periodos_por_tarefa, algoritmo, ctx_time, quantum=None, eficiencia=None, protocolo=None, alpha=None)` -> `ResultadoSimulacao` — calcula $tt/tw/1^a$ execução por tarefa e as médias, e embrulha tudo (junto com os parâmetros recebidos) num `ResultadoSimulacao`. Chamado por todo algoritmo, ao final da própria simulação.

Derivação pro gráfico (`algoritmos/_derivar_grafico.py`), todas recebendo um `TarefaResultado` já montado:

- `calcular_esperas(tr)` -> `list[tuple[float, float]]` — os intervalos em que a tarefa não está coberta por nenhum `Periodo` (parada na fila, sem estar bloqueada).
- `calcular_recursos_em_uso(tr)` -> `list[tuple[Recurso, float, float]]` — pra cada `Recurso` da tarefa, o intervalo absoluto (início/fim) em que ela o reivindica, calculado a partir de quando seu tempo de execução própria cruza `Recurso.inicio` e `Recurso.inicio + duracao`.

- `calcular_preempcoes(tr) -> list[float]` — os instantes em que uma execução termina e outra coisa de fato ocupou o processador antes da próxima.

Validação compartilhada (`algoritmos/validacoes.py`):

- `validar_quantum(ctx_time, quantum)` — levanta `ErroValidacao` se o quantum não for maior que o custo de troca de contexto.

Persistência de cenário (`views/cenario_io.py`):

- `validar_cenario(cenario: dict) -> bool` — confere se um dict lido de um `.json` tem o formato esperado.
- `abrir_cenario_de_arquivo() -> tuple[dict | None, str | None]` — abre o diálogo nativo de arquivo, lê e valida; devolve o cenário, ou uma mensagem de erro, ou `(None, None)` se cancelado.

6. Parâmetros configuráveis

Parâmetro	Onde se aplica	Faixa válida	Valor padrão
Tempo de troca de contexto (<code>ttc / ctx_time</code>)	Todos os algoritmos	≥ 0	0
Quantum (<code>tq / quantum</code>)	Round-Robin	$> ttc$	0 (precisa ser preenchido)
Fator de envelhecimento (α / <code>alpha</code>)	PRIOc	≥ 0 (0 desliga o envelhecimento)	0
Protocolo de correção	PRIOp	Nenhum, Herança ou Teto	Nenhum

7. Funcionamento interno

Não há um único laço comum — os seis algoritmos se dividem em dois grupos, por como avançam o tempo.

Sem preempção (FCFS, SJF, PRIOc) — cada tarefa escolhida roda até o fim de uma vez, sem parar no meio:

- **FCFS** não tem nem fila: um `for` simples sobre as tarefas já ordenadas por chegada.
- **SJF** e **PRIOc** usam `while tarefas_pendentes:` — a cada volta, montam a fila de prontas (`chegada <= tempo_atual`), despacham a primeira e a rodam até terminar, removendo-a de `tarefas_pendentes`. **PRIOc** difere num ponto: antes de montar a fila, recalcula a prioridade efetiva de toda tarefa pendente ($\text{prioridade_base} + \alpha \times \text{tempo_de_espera}$, contado desde a chegada) e a reconstrói do zero a cada volta (envelhecimento).

Com preempção (SRTF, Round-Robin, PRIOp) — o laço é `while ids_ao_finalizados:` cada volta roda só uma **fatia** da tarefa escolhida, limitada pelo primeiro evento que a interrompe: a próxima chegada (SRTF, PRIOp), o fim do quantum (Round-Robin) ou um evento de recurso (só PRIOp). O restante do `tp` fica pendente pra ser retomado numa volta futura.

Em ambos os grupos, a troca de contexto (`Periodo` do tipo `TROCA_CONTEXTO`) só é cobrada quando a tarefa despachada é diferente da que rodou por último, e seu custo é somado antes da fatia de execução, não descontado dela.

PRIOp acrescenta um checkpoint antes de despachar: se a candidata precisa, naquele instante, de um recurso que outra tarefa pendente já está usando, ela não é despachada — abre-se (ou mantém-se) um `Periodo` do tipo `BLOQUEIO_DIRETO` para ela, e a próxima candidata da fila é avaliada no lugar. Sob **Herança**, a prioridade de quem segura o recurso sobe pra da tarefa bloqueada, se for maior. Sob **Teto**, a prioridade de quem obtém o recurso já sobe pro maior valor entre todas as tarefas que algum dia o disputam. Nos dois casos, a prioridade volta ao original assim que o recurso é solto.

8. Convenções de simulação

- **Unidade de tempo:** segundo. Todos os campos de entrada (`chegada` , `tp` , `quantum` , `ttc` , `inicio/duracao` de um recurso) aceitam tanto inteiro quanto fracionário de segundo — a tela aceita decimal (vírgula) em qualquer um deles, e o motor de simulação suporta aritmética de ponto flutuante.
- **Critério de desempate** entre tarefas equivalentes: menor `chegada` , e caso empate, menor `id` primeiro.
- **Cobrança da troca de contexto:** só ocorre quando a tarefa que assume o processador é **diferente** da que rodou por último. O custo é **somado** à fatia de execução seguinte — o relógio avança `ttc` antes de a execução em si começar — e não é descontado do `tp` da tarefa.
- **Posição de retorno à fila de uma tarefa preemptada** — para os preemptáveis:
- **SRTF:** a fila persiste entre iterações (não é reconstruída) — a tarefa preemptada reentra pela mesma regra de inserção por `tp` usada pra qualquer pendente, disputando posição normalmente.
- **Round-Robin:** vai sempre pro **fim** da fila, sem comparação — FIFO.
- **PRIOp:** reinserida pela prioridade.
- **Sentido da escala de prioridades:** **maior** valor numérico é **mais** prioritário (o oposto da convenção *nice* do Unix).
- **Referência de tempo do uso de um recurso:** `Recurso.inicio` e `Recurso.duracao` são medidos no tempo de processamento **próprio** da tarefa (quanto ela mesma já executou), não no tempo absoluto da simulação. `inicio = 1` significa "depois que essa tarefa já rodou 1 segundo de CPU", não "no instante `t=1`".

9. Dependências e ambiente

- Python 3.11 (desenvolvido e testado em 3.11.9).
- Interface gráfica: Tkinter, biblioteca padrão do Python.
- Única dependência externa: **matplotlib** (usada só para desenhar o diagrama de tempo).

- O executável `Escalonadores.exe` foi gerado com **PyInstaller** (`pyinstaller main.py --name Escalonadores --onefile --windowed`).